CHAPTER 06

Terraform Modules

Organize, reuse, and scale your infrastructure code

- What & Why
- Build & Use
- Registry Modules
- Best Practices
- Scenario

registry.terraform.io

06.1

What Are Modules & Why Use Them?

A Terraform module is a **container for multiple resources** managed as a single unit.

Reuse Code

Stop rewriting the same Terraform configurations for every project.

Simplify Complexity

Organize infrastructure into smaller, manageable pieces.

Ensure Consistency

Keep Dev, Staging, and Production environments aligned.

Every Terraform configuration is a module — even a single main.tf file.

Modules live in folders with their own main.tf, variables.tf, and outputs.tf.

Instructor note: emphasize that modularity is the same principle as functions in programming — inputs, logic, outputs.

06.2

Creating & Using Modules — Project Structure

Step 1 — Create a Reusable EC2 Module

```
terraform-project/
├── main.tf                 # Calls the module
├── variables.tf            # Defines input variables
├── outputs.tf              # Defines output variables
├── modules/
│   ├── ec2-instance/       # Custom EC2 module
│   │   ├── main.tf         # Module definition
│   │   ├── variables.tf    # Module variables
│   │   ├── outputs.tf      # Module outputs
```

Standard Module Layout

- Root config calls the module and wires up variables/outputs
- modules/ec2-instance/ is a self-contained, reusable unit
- Each module folder mirrors the root: main, variables, outputs

Talking point: walk through the folder tree live in an editor before showing any code, so learners see where each file lives.

06.2

Define the EC2 Module

Step 2 — modules/ec2-instance/main.tf

```
resource "aws_instance" "ec2" {
  ami           = var.ami
  instance_type = var.instance_type

  tags = {
    Name        = var.instance_name
    Environment = var.environment
  }
}
```

What this defines

- One aws_instance resource, the module's core building block
- ami and instance_type are parameterized — not hardcoded
- Tags merge a fixed Name with the caller's environment value

This is the module's logic — the same pattern as a function body.

Define Variables for the Module

Step 3 — modules/ec2-instance/variables.tf

```
variable "ami" {
  description = "The AMI ID for the EC2 instance"
  type        = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.micro"
}
```

Inputs = the module's API

- ami and instance_name have no default — callers must supply them
- instance_type defaults to t2.micro, but can be overridden
- environment is required, keeping tagging consistent everywhere

Full variables.tf also declares instance_name and environment (both required strings).

Define Outputs for the Module

Step 4 — modules/ec2-instance/outputs.tf

```
output "public_ip" {
  description = "Public IP of the EC2 instance"
  value       = aws_instance.ec2.public_ip
}

output "instance_id" {
  description = "EC2 Instance ID"
  value       = aws_instance.ec2.id
}
```

Passing data back up

- Outputs expose values from inside the module to the caller
- public_ip and instance_id become module.web_server.public_ip, etc.
- Useful for chaining modules or printing after apply

Use the Module in the Main Configuration

Step 5 — main.tf (root configuration)

```
provider "aws" {
  region = "us-east-1"
}

module "web_server" {
  source         = "./modules/ec2-instance"
  ami            = "ami-0c55b159cbfafe1f0"
  instance_type  = "t3.micro"
  instance_name  = "WebServer"
  environment    = "staging"
}
```

Calling the module

- source points at the local module folder
- Every variable.tf input is supplied as an argument
- Instantiates one module block = one deployed instance

Ask learners: what changes if we call the module a second time with a different name?

06.2

Apply the Configuration

Step 6 — run Terraform from the project root

```
$ terraform init
$ terraform apply -auto-approve
```

Result: Terraform deploys an EC2 instance using the module.

terraform init downloads providers and registers the module; apply builds the plan and provisions resources.

Demo Tips

- Run terraform plan first to preview changes
- Point out the module.web_server prefix in plan output
- Show public_ip / instance_id outputs after apply

06.3

Using Public Terraform Registry Modules

Terraform provides a **public registry** of prebuilt modules for common infrastructure needs.

Browse modules at registry.terraform.io

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "5.0.0"

  name    = "my-vpc"
  cidr    = "10.0.0.0/16"

  tags = {
    Terraform   = "true"
    Environment = "staging"
  }
}
```

Registry Advantage

- Skip writing a VPC module from scratch
- source references org/module/provider on the registry
- version pins a stable, tested release

This automatically provisions a VPC without writing custom code.

06.4

Module Best Practices

Use Versioning

Pin a version when consuming public modules for stability.

Keep Code DRY

Don't Repeat Yourself — extract shared logic into modules.

Meaningful Names

Choose clear, descriptive variable and output names.

Separate Folders

Store each module in its own dedicated folder.

Document Inputs/Outputs

Add descriptions so consumers understand the module's API.

06.0

Scenario: Reusable EC2 Module for Multiple Environments

main.tf — calling the same module twice

```
provider "aws" {
  region = "us-east-1"
}

module "staging_server" {
  source         = "./modules/ec2-instance"
  ami            = "ami-0c55b159cbfafe1f0"
  instance_type  = "t3.micro"
  instance_name  = "StagingServer"
  environment    = "staging"
}

module "production_server" {
  source         = "./modules/ec2-instance"
  ami            = "ami-0c55b159cbfafe1f0"
  instance_type  = "t3.large"
  instance_name  = "ProductionServer"
  environment    = "production"
}
```

One Module, Two Sizes

- Same source module used for both environments
- instance_type scales up for production (t3.micro → t3.large)
- environment tag keeps resources clearly separated

This is the payoff of modules: one definition, many consistent deployments.

SCENARIO

Apply Terraform

```
$ terraform init
$ terraform apply -auto-approve
```

Result: Terraform deploys two instances — one for Staging, one for Production.

Both instances come from the same module source, so future fixes or improvements apply to every environment consistently.

Class Exercise

- Add a third module block for a QA environment
- Change QA's instance_type to t3.small
- Run terraform plan and compare all three

Chapter 6 Summary

Terraform Modules

What modules are & why they matter

Creating and using a custom module

Using public Terraform Registry modules

Module best practices

Scenario: one module, multiple environments

Next: Chapter 7 — Remote State & Backends